Habib University

Habib University

OBJECT-ORIENTED PROGRAMMING FALL 2025

Week 03

Pointers and Dynamic Memory Allocation

THIS WEEK'S AGENDA

01. Pointers in C++
02. Declaring Pointer Variables
03. Pointers and Functions
04. Reference Variables in C++
05. Pass by Pointer vs Pass by Reference
06. Arrays and Pointers
07. Static vs Dynamic Arrays
08. Double Pointers

Pointers in C++

POINTERS IN C++

- **Every byte in a computer's memory has an address**.
- In C++ a pointer variable stores the address (or the memory location) of a variable.
- Let's say we declare an int:
  - `int x = 23;`
  - The variable x has the value 23, and is allocated 4 bytes of memory. But what is its memory address?
  - This address can be found by using a pointer variable in the following way:
    - `int *p = &x;`

```
int x = 23;
int* p = &x;
```

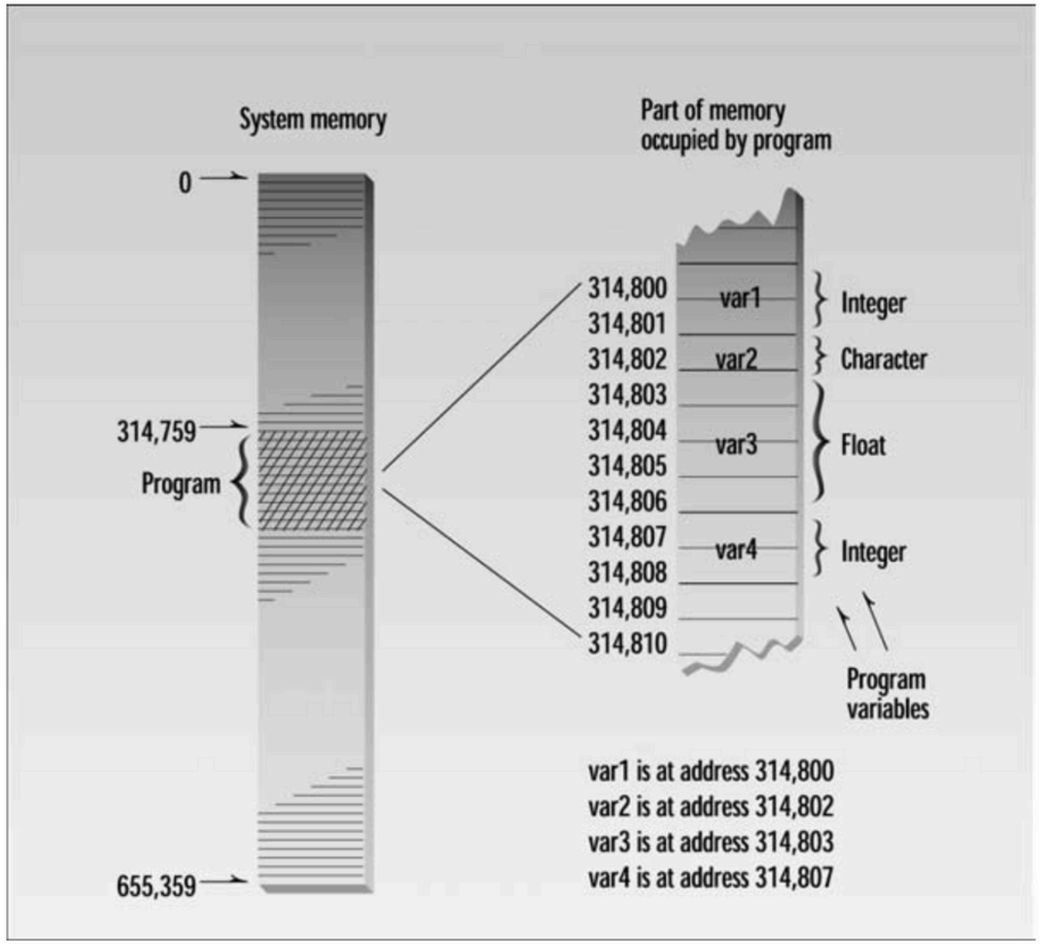


FIGURE 10.1
Memory addresses.

POINTERS IN C++

- In C++, a pointer to a data type is declared as:
  - `type *var_name`
- Therefore, a pointer to an integer variable would be declared as:
  - `int *p;`

```
int a = 9, b;
int *p = &a;
b = *p;
std::cout << "a = " << a << std::endl;
std::cout << "p = " << p << std::endl;
std::cout << "*p = " << b << std::endl;
```

```
a = 9
p = 0x16af02a38
*p = 9
```

MEMORY ADDRESSES

- Memory addresses are **not** the same when you rerun the program, nor across different machines.

```cpp
#include <iostream>

int main() {
    int i = 4, j = 6, k = -2;
    std::cout << i << ", " << j << ", " << k << std::endl;

    // Printing the addresses of the variables
    std::cout << &i << ", " << &j << ", " << &k << std::endl;
    return 0;
}
```

DECLARATIONS AND INITIALIZATIONS

- What do you think would be the output of:
  - `double x = 5.5;`
  - `cout << *(&x);`
- In C++, we need to declare the type of the variable the pointer is pointing to. Therefore, a pointer to a character is an address to a memory location having 1 byte, a pointer to an integer having 4 bytes and so on.
- In general, we can have pointers for integers (int* p), characters (char* p), doubles (double* p), and so on.

Lets demo this live!

DECLARATIONS AND INITIALIZATIONS

```cpp
#include <iostream>

int main() {
    int i = 5;
    int j = i;
    std::cout << i << ", " << j << std::endl;
    std::cout << &i << ", " << &j << std::endl;
    return 0;
}
```

Would &i and &j return the same address?

DECLARATIONS AND INITIALIZATIONS

- So in essence, pointers point to a memory address.
- Where are pointers stored then? Are they stored in the memory too?

```cpp
#include <iostream>

int main() {
    int i = 5;
    int j = i;
    int *p = &i;
    std::cout << i << ", " << j << std::endl;
    std::cout << &i << ", " << &j << std::endl;
    std::cout << p << std::endl;
    std::cout << &p << std::endl;
    return 0;
}
```

What would the last line output?

Pointers And Functions

POINTERS AND FUNCTIONS

- Typically variables are passed **by value** to functions in C++ which means that a ***copy*** of the variable is passed to the function.
- Modifications to the passed parameter does not change the value of the argument. Consider the following program:
  - Is the value of i 15 or 5?

```cpp
#include <iostream>

void func(int x) {
    x += 10;
}

int main() {
    int i = 5;
    func(i);
    std::cout << i << std::endl;
    return 0;
}
```

PASSING BY POINTERS

- Using pointers, you can pass a variable using its ***memory address*** instead of the value.
- Changes made to the value pointed to by the address changes the value of the argument too.
- Consider the following program and note the syntax carefully:
  - Now what is the value of i?

```cpp
#include <iostream>

void func(int *x) {
    *x += 10;
}

int main() {
    int i = 5;
    func(&i);
    std::cout << i << std::endl;
    return 0;
}
```

Reference Variables in C++

REFERENCE VARIABLES IN C++

- A reference variable in C++ acts as an alias for another already existing variable.
- Once a reference is initialized with a variable, either the variable name or the reference name may be used to refer to the variable. For example:
  - `double j = 2.5;`
  - `double &k = j;`
- Here j and k refer to the same variable and hence k is a reference to j.
- Any changes made in k are also reflected in j.

REFERENCE VARIABLES IN C++

- There are a few differences between references and pointers in C++:
  a. References cannot be NULL, it should always be referring to an existing piece of storage/memory. However, pointers can be NULL (nullptr).
  b. References cannot be changed to refer to another object once initialized. However, pointers can point to another object at any time.
  c. A reference must be initialized when it is created, but pointers can be initialized at any time.

REFERENCE VARIABLES IN C++

Parameters	Pass by Pointer	Pass by Reference
Passing Arguments	We pass the address of arguments in the function call.	We pass the arguments in the function call.
Accessing Values	The value of the arguments is accessed via the dereferencing operator *	The reference name can be used to implicitly reference a value.
Reassignment	Passed parameters can be moved/reassigned to a different memory location.	Parameters can't be moved/ reassigned to another memory address.
Allowed Values	Pointers can contain a NULL value, so a passed argument may point to a NULL or even a garbage value.	References cannot contain a NULL value, so it is guaranteed to have some value.

PASSING BY REFERENCE

- Another way of passing parameters to functions is passing by reference.
- Doing this creates a reference to the parameter that is supposed to be passed.
- Consider the following program and note the syntax carefully:
  - What is the value of i?

```cpp
#include <iostream>

void func(int &x) {
    x += 10;
}

int main() {
    int i = 5;
    func(i);
    std::cout << i << std::endl;
    return 0;
}
```

Arrays And Pointers

ARRAYS AND POINTERS IN C++

- **Recall**: An array is a collection of homogeneous data types in C++.
- The memory address of the first element stored in the array is referred to by the name of the array. In essence, the **name of the array** is itself an **address** and can be treated as a pointer.

```cpp
#include <iostream>

int main() {
    bool arr[] = {true, false, true, false, true};
    std::cout << arr << std::endl;
    for (int index = 0; index < 5; index++) {
        std::cout << *(arr + index);
    }
    return 0;
}
```

ARRAYS AND POINTERS IN C++

- Using a pointer to an array in C++.

```cpp
#include <iostream>

int main() {
    int arr[] = {12, 25, 13, 432, -5};
    int *arrptr = arr;
    for (int i = 0; i < 5; i++) {
        std::cout << *(arrptr + i) << " ";
    }
    return 0;
}
```

Static And Dynamic Arrays

STATIC VS DYNAMIC ARRAYS

- We have encountered only static arrays so far that are initialized at compile time. The size for these arrays is fixed, and they automatically get deleted when the function defining them completes execution.
- Memory for static arrays is allocated on the ***stack***.
- Examples of static arrays are:
  - `int b[10];`
  - `bool x[2];`
  - `char str[210];`

DYNAMIC ARRAYS

- Dynamic arrays allow us to allocate memory dynamically and these are particularly useful when the size of the array is unknown at compile time.
- A dynamic array can be created using the “new” keyword and can be deleted using the “delete” keyword. They are initialized in the following way:
  - `int *arr = new int[n]`
- A dynamic array is initialized at run time on the heap and they have to be explicitly deallocated using the “delete” keyword.

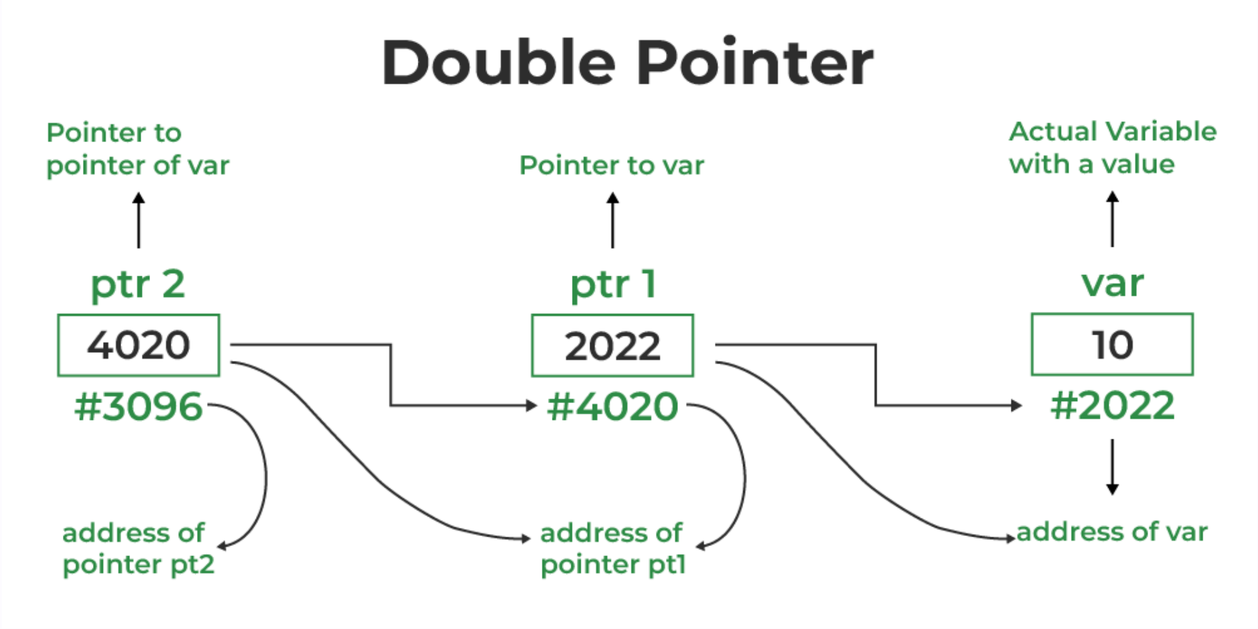
DYNAMIC ARRAYS

```cpp
#include <iostream>

int main() {
    // initializing a dynamic array
    int size = 5;

    int* dynamicArray = new int[size];

    for (int i = 0; i < size; i++) {
        dynamicArray[i] = i * 2;
    }

    // Don't forget to delete the array later
    delete[] dynamicArray;
    return 0;
}
```

Double Pointers

DYNAMIC ARRAYS

- Double pointers in C++ can be very tricky to understand. They are used, among other things, to refer to 2D arrays which are essentially an array of arrays.



DYNAMIC ARRAYS

- Consider the following:
  - `int rows = 3, cols = 4;`
  - `int **arr = new int*[rows];`
- This declares an array of pointers.

DYNAMIC ARRAYS

```cpp
#include <iostream>

int main() {
    int rows = 3, cols = 4;
    // Step 1: Declare an array of pointers
    int **arr = new int*[rows];
    // Step 2: Allocate memory for each row
    for (int i = 0; i < rows; ++i) {
        arr[i] = new int[cols];
    }
    // Step 3: Initialize the array using pointer arithmetic
    for (int i = 0; i < rows; ++i) {
        for (int j = 0; j < cols; ++j) {
            *(*(arr + i) + j) = i * cols + j;
        }
    }
```

DYNAMIC ARRAYS

```
17        // Step 4: Print the array
18        for (int i = 0; i < rows; ++i) {
19            for (int j = 0; j < cols; ++j) {
20                std::cout << *(*(arr + i) + j) << " ";
21                // Equivalent to arr[i][j]
22            }
23            std::cout << std::endl;
24        }
25        // Step 5: Deallocate memory
26        for (int i = 0; i < rows; ++i) {
27            delete[] arr[i]; // Free each row
28        }
29        delete[] arr; // Free the array of pointers
30        return 0;
31    }
```

DOUBLE POINTERS AND STRINGS

```cpp
#include <iostream>

int main() {
    // Step 1: Declare an array of cstrings (array of char pointers)
    const char* arr[] = {"Hello", "World", "C++", "Programming"};
    // Step 2: Declare a pointer to point to the array
    const char** p = arr;
    // Step 3: Access the strings using the double pointer
    std::cout << "First string: " << *p << std::endl; // Outputs: Hello
    std::cout << "Second string: " << *(p + 1) << std::endl; // Outputs: World
    std::cout << "Third string: " << *(p + 2) << std::endl; // Outputs: C++
    std::cout << "Fourth string: " << *(p + 3) << std::endl; // Outputs: Programming
    // Step 4: Access individual characters in the string using pointer arithmetic
    std::cout << "First character of second string: " << **(p + 1) << std::endl; // Outputs: o
    std::cout << "Third character of the first string: " << *(*(p) + 2) << std::endl; // Outputs: l
    return 0;
}
```

Summary

SUMMARY

- Pointers can be used to pass addresses to individual variables.
- We may also pass a pointer to an array.
- Passing by reference vs Passing by pointers.
- Array names are used to refer to the memory address of the first element in the array.
- Double pointers can be used to point to a pointer variable. They can be used for 2D arrays in C++.

READINGS

- Object-Oriented Programming in C++ by Robert Lafore - Chapter 05, 07 and 10

ACKNOWLEDGEMENTS

Many of the slides of this lecture have been taken/modified from the lecture notes of:

- Object Oriented Programming Fall 2024 by Dr. Faisal Alvi, Assistant Professor, Computer Science
- Object Oriented Programming Fall 2023 by Dr. Musabbir Majeed, Assistant Professor, Computer Science